

SAIKAT DAS

🌐 saikat.in | ✍️ blog.saikat.in
🔗 LinkedIn | 🐙 GitHub | 📄 LeetCode

☎️ +91 6202051066
✉️ sd.saikat369@gmail.com
📍 Varanasi, India

EDUCATION

Indian Institute of Technology (BHU), Varanasi
Bachelor of Technology in Mining Engineering; CGPA: 8.39/10.0

Varanasi, India
Jul 2023 – May 2027

TECHNICAL SKILLS

Languages & Frameworks	Go, Python, C++ , Dart, FastAPI, Flutter, React
Backend & Agentic AI	REST APIs, gRPC, Microservices, Kafka, LLMs/VLMs, Agentic Workflows
Databases & Infra	PostgreSQL, Redis, Apache Solr, Memgraph, Docker, Linux, Git, Grafana k6

EXPERIENCE

Software Engineering Intern | *COPIOUS · Tripple Double* Jun 2026 – Jul 2026

- Built a **Flutter** split-screen web portal with auto-save and rule-based validation for FIBA scoresheet digitization.
- Engineered a **VLM** OCR pipeline via **OpenRouter**, A/B testing models to optimize accuracy vs. cost.
- Reduced hallucination and token costs via dynamic prompts, **JSON key aliasing**, and output-embedded reasoning.
- Cut operator cost by **91.92%** (\$0.52 → \$0.042) and entry time by **73.33%** (30m → 8m) via pre-filled entry.

Flutter Developer Intern | *Vrajpath iTech LLP* Dec 2024 – Jan 2025

- Reduced API requests by ~**30%** via **Riverpod** state caching, validated in **Dart DevTools**.
- Integrated **Razorpay** for secure payments and **Google Maps** for real-time geolocation.

PROJECTS

TradeOrders – Order Matching Engine | *Go, Kafka, PostgreSQL, Redis, Docker* | 🐙 [dis70rt/TradeOrders](#)

Solved lock contention in high-volume trading via an event-driven architecture, scaling throughput 18x.

- Decoupled API from DB via **Kafka**, eliminating all I/O bottlenecks; scaled throughput to **18.8k req/s** (0 errors).
- Partitioned Kafka topics by instrument for **lock-free**, sequential matching, enabling concurrent process per symbol.
- **Grafana k6** load tested at **7,624 VUs**: hit **18.2k req/s** sustained throughput with a **231ms p95 latency**.

Bluppi – Shared Music Playback | *Go, gRPC, PostgreSQL, Redis, Solr, Flutter* | 🐙 [backend](#) / [frontend](#) | 🌐 [bluppi.saikat.in](#)

A real-time synchronization engine that fixes network lag across distributed devices.

- Designed **PTP-style** clock synchronization with **quorum-based** buffering for consistent playback across devices.
- Tracks online presence via a **gRPC** gateway: **51k** concurrent connections with 0 errors and **p99 15ms** latency.
- Live ephemeral chat via **Redis Pub/Sub**, social features via **Memgraph**, and a 500GB music catalog via **Solr**.

QQL (Quick Query Language) – Approximate Query Processing Shell | *Python, PostgreSQL* | 🐙 [dis70rt/qql](#)

A shell sitting above PostgreSQL that accelerates massive dataset aggregations 10x using bounded statistical error.

- Designed a dynamic **pilot query** engine to estimate statistical sample sizes, supporting **COUNT**, **SUM**, **AVG** functions.
- Benchmarked against exact PostgreSQL queries: achieved a **10x speedup** at 5% error and **6x** at 1% error bound.
- Awarded **Special Mention** at the **e6Data Hackathon 2024** for innovative approaches to database acceleration.

SubPaper – Reddit Wallpapers App | *Flutter* | 🐙 [dis70rt/subpaper](#) | 🌐 [subpaper.saikat.in](#) | ▶️ [Play Store \(Archived\)](#)

A mobile application that aggregates and caches high-res wallpapers directly from Reddit.

- Published Reddit wallpaper app to Play Store. **300+ downloads**, **4.5/5 rating**.
- Implemented on-device caching for fast content delivery and integrated **AdMob** for monetization.

ACHIEVEMENTS

Won **1st Place** at the **BNY Hackathon**, building a solution for **transaction processing & settlement**. Apr 2026

Received a **Special Mention** at the **e6Data Hackathon** for building a **QQL** system. Oct 2025

Secured **2nd Place** at the **Nutanix Hackathon** by building **WizFlow**, a **real-time workflow engine**. Apr 2025

Ranked in the **Top 9.22%** globally on **LeetCode** with a **1,778** peak rating and **256+** problems solved. Present

Achieved **Pupil** rank (**1,217** Max Rating) on **Codeforces**, solving **150+** algorithmic challenges. Present